
DSC 140B - Homework 08

Due: Wednesday, March 4

Instructions:

- Write your solutions to the following problems **by hand**, either on another piece of paper that you scan or using a tablet. Typed solutions will not be accepted for credit!
 - Code listings are an exception. You do not need to handwrite code, and you can instead include the code as a screenshot.
- Unless otherwise noted by the problem's instructions, show your work or provide some justification for your answer.
- Homework problems are graded pass/fail on completeness and effort, not correctness.
- Homeworks are due via Gradescope at 12:00 AM.

Problem 1. (3 credits)

As a data scientist you will have the opportunity to work on problems that are of great importance to society. This is not one of those problems.

The *menu-match* dataset consists of 646 images of food from three different restaurants: an Asian restaurant, an Italian restaurant, and a soup restaurant. The data set was constructed by employees of Microsoft Research¹.

The data set is available at the following link:

<https://f000.backblazeb2.com/file/dsc-data/menu-match.npz>

The file is in compressed numpy format; it can be loaded with 'np.load'. Once loaded, it behaves like a dictionary with four keys: `X_train`, `X_test`, `y_train`, and `y_test`, corresponding to the training data, test data, training labels, and test labels, respectively. For example, to get the training data:

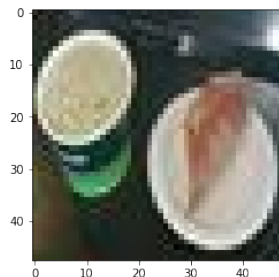
```
>>> data = np.load("menu-match.npz")
>>> data['X_train']
```

The training data is a 480 x 48 x 48 x 3 tensor, since it consists of 480 images, each 48 x 48 pixels with 3 colors.

Note that the given labels are strings: “a” for Asian restaurant, “i” for Italian, and “s” for soup.

You may display an image with `matplotlib` using `plt.imshow`. For example:

```
>>> # plot test example #30
>>> plt.imshow(data['X_test'][30])
```



¹<http://neelj.com/projects/enumatch/>

- a) Using PyTorch, train a convolutional neural network to predict whether a given image is from the Asian restaurant or not (thus turning the problem into a binary classification problem). You will need to determine the network architecture, but the example code given in lecture is a good starting point. Report the test accuracy – your model should be able to get above 70% of the test set correct (preferably higher!). Show your code.

In this section, you should not use any pre-trained models or transfer learning; you should train the network from scratch.

Solution: We train a simple CNN with three convolutional layers (with 32, 64, 128 filters) each followed by batch normalization, ReLU, and max pooling, then a fully-connected classifier. We use BCEWithLogitsLoss and Adam as the optimizer.

Test accuracy was around 87% – quite a bit higher than the 70% threshold!

```
"""
Part (a): Train a CNN from scratch for binary classification (Asian vs. Not Asian)
"""

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
import urllib.request
import os

# Download and load data

DATA_URL = "https://f000.backblazeb2.com/file/dsc-data/menu-match.npz"
DATA_PATH = "menu-match.npz"

if not os.path.exists(DATA_PATH):
    print("Downloading dataset...")
    urllib.request.urlretrieve(DATA_URL, DATA_PATH)

data = np.load(DATA_PATH)
X_train_raw = data["X_train"] # (480, 48, 48, 3)
X_test_raw = data["X_test"]
y_train_raw = data["y_train"] # strings: 'a', 'i', 's'
y_test_raw = data["y_test"]

# Convert to binary labels: 1 if Asian ('a'), 0 otherwise
y_train = (y_train_raw == "a").astype(np.float32)
y_test = (y_test_raw == "a").astype(np.float32)

# Normalize pixel values to [0, 1]; similar to standardizing, this helps condition the
# optimization process and can lead to faster convergence.
X_train = X_train_raw.astype(np.float32) / 255.0
X_test = X_test_raw.astype(np.float32) / 255.0

# As-is, the images are in (N, H, W, C) format (batch size, height, width, channels).
# PyTorch's Conv2d expects (N, C, H, W), so we need to transpose the axes.
X_train = np.transpose(X_train, (0, 3, 1, 2))
X_test = np.transpose(X_test, (0, 3, 1, 2))

print(f"Training samples: {len(y_train)}, Test samples: {len(y_test)}")
print(
    f"Asian (train): {y_train.sum():.0f}, Not Asian (train): {(1 - y_train).sum():.0f}"
)
```

```

)

# Model

# we will train a simple CNN from scratch. As usual with a CNN, we'll break the
# architecture into two parts: 1) a "feature extractor" that uses convolutional layers
# to learn spatial features, and 2) a "classifier" that uses fully connected layers to
# make the final prediction. We'll store the first part in `self.features` and the
# second part in `self.classifier`.

# you can achieve good results with different architectures, but here is a simple one
# that works well. We will use:
#
# - 3 convolutional layers with increasing number of filters (32, 64, 128). Each
#   filter will be of size 3x3 (this is pretty standard for small images).
# - Batch normalization and ReLU activation after each convolution
# - Max pooling after each convolution to reduce spatial dimensions

class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2), # 48 -> 24
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2), # 24 -> 12
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(2), # 12 -> 6
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 6 * 6, 128),
            nn.ReLU(),
            nn.Linear(128, 1),
            # we will use a sigmoid activation at inference time, but note that we don't
            # apply it here! That is because we will use `BCEWithLogitsLoss` which
            # combines the sigmoid activation and binary cross-entropy loss into one
            # function that is more numerically stable.
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x.squeeze(1)

# Training

train_tensor_x = torch.tensor(X_train)
train_tensor_y = torch.tensor(y_train)
test_tensor_x = torch.tensor(X_test)
test_tensor_y = torch.tensor(y_test)

```

```

# we don't *have* to use mini-batch SGD here (we could just use one full batch), but
# let's set it up so that you have an example to use if you want to implement SGD in the
# future. To perform SGD with minibatches, we need to create a DataLoader that will
# yield batches of data during training. We will use a batch size of 32, which is a
# common choice for small datasets.
train_dataset = TensorDataset(train_tensor_x, train_tensor_y)
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)

model = SimpleCNN()
criterion = nn.BCEWithLogitsLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(50):
    model.train()
    total_loss = 0

    # loop over the batches and take one step of optimization for each batch
    for xb, yb in train_loader:
        optimizer.zero_grad()
        preds = model(xb)
        loss = criterion(preds, yb)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * len(xb)

    # this code prints the training progress. It is somewhat optional, but generally
    # a good idea to do something like this. We'll only print out an update every 5
    # epochs to avoid cluttering the output.
    if (epoch + 1) % 5 == 0 or epoch == 0:
        model.eval()

        # torch.no_grad() tells PyTorch that we are not going to compute gradients
        # during this block of code. This helps save memory and can speed up
        # computations since PyTorch doesn't have to keep track of the operations for
        # backpropagation.
        with torch.no_grad():
            test_preds = model(test_tensor_x)
            test_pred_labels = (test_preds > 0).float()
            acc = (test_pred_labels == test_tensor_y).float().mean().item()
        print(
            f" Epoch {epoch+1:3d} | Loss: {total_loss/len(train_dataset):.4f} | Test Acc: {acc:.4f}"
        )

# Final accuracy
model.eval()
with torch.no_grad():
    test_preds = model(test_tensor_x)
    test_pred_labels = (test_preds > 0).float()
    acc = (test_pred_labels == test_tensor_y).float().mean().item()

print(f"\nFinal test accuracy (CNN from scratch): {acc:.4f} ({acc*100:.1f}%)")

```

- b) Now use a pre-trained model (e.g. ResNet) and transfer learning to solve the same problem. The model you use is up to you; ResNet-18 is a good starting choice. Report your code and your results.

Solution: We use a pre-trained ResNet-18 as a fixed feature extractor. Below are two versions of this approach. The first approach simply “freezes” the convolutional layers and retrain the final

fully-connected layer. This will work in principle, but it is slow. The problem is that each step of the optimizer requires a forward pass where we evaluate the network. Since the convolutional layers are all frozen, this part of the computation never changes – it's only at the last layer where the parameters are updated and the prediction changes. Therefore, we do a lot of redundant computation by evaluating the convolutional layers at every step.

```

"""
Part (b): Transfer learning with pre-trained ResNet-18 for binary classification
(Asian vs. Not Asian)
"""

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
import torchvision.models as models
import torchvision.transforms as transforms
import urllib.request
import os

# Download and load data

DATA_URL = "https://f000.backblazeb2.com/file/dsc-data/menu-match.npz"
DATA_PATH = "menu-match.npz"

if not os.path.exists(DATA_PATH):
    print("Downloading dataset...")
    urllib.request.urlretrieve(DATA_URL, DATA_PATH)

data = np.load(DATA_PATH)
X_train_raw = data["X_train"] # (480, 48, 48, 3)
X_test_raw = data["X_test"]
y_train_raw = data["y_train"] # strings: 'a', 'i', 's'
y_test_raw = data["y_test"]

# Convert to binary labels: 1 if Asian ('a'), 0 otherwise
y_train = (y_train_raw == "a").astype(np.float32)
y_test = (y_test_raw == "a").astype(np.float32)

# Normalize pixel values to [0, 1]
X_train = X_train_raw.astype(np.float32) / 255.0
X_test = X_test_raw.astype(np.float32) / 255.0

# Convert from (N, H, W, C) to (N, C, H, W) for PyTorch
X_train = np.transpose(X_train, (0, 3, 1, 2))
X_test = np.transpose(X_test, (0, 3, 1, 2))

print(f"Training samples: {len(y_train)}, Test samples: {len(y_test)}")
print(
    f"Asian (train): {y_train.sum():.0f}, Not Asian (train): {(1 - y_train).sum():.0f}"
)

# Prepare data for ResNet

# We will use ResNet-18 as our pre-trained model.

# ResNet was trained on ImageNet, which uses 224x224 images normalized with specific
# per-channel means and standard deviations. This means that ResNet-18 expects its

```

```

# input images to be 224x224 and normalized in the same way, so we will
# resize our 48x48 images to 224x224 and apply the same normalization.

train_tensor_x = torch.tensor(X_train)
train_tensor_y = torch.tensor(y_train)
test_tensor_x = torch.tensor(X_test)
test_tensor_y = torch.tensor(y_test)

# these are the mean and std values used for normalizing ImageNet images.
imagenet_mean = torch.tensor([0.485, 0.456, 0.406]).view(1, 3, 1, 1)
imagenet_std = torch.tensor([0.229, 0.224, 0.225]).view(1, 3, 1, 1)

resize = transforms.Resize((224, 224), antialias=True)

X_train_resized = resize(train_tensor_x)
X_test_resized = resize(test_tensor_x)

# standardize using ImageNet mean and std
X_train_resized = (X_train_resized - imagenet_mean) / imagenet_std
X_test_resized = (X_test_resized - imagenet_mean) / imagenet_std

train_dataset = TensorDataset(X_train_resized, train_tensor_y)
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)

# Model

# Load a pre-trained ResNet-18. This model has already learned useful visual features
# (edges, textures, shapes, etc.) from ImageNet. We replace the final fully-connected
# layer with a single output for our binary classification task.
resnet = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
resnet.fc = nn.Linear(resnet.fc.in_features, 1)

# Freeze all layers except the final fc layer. This way, we only train the new
# classifier head and keep the pre-trained feature extractor fixed. We do this by
# telling PyTorch not to compute gradients for the pre-trained layers.
for param in resnet.parameters():
    param.requires_grad = False
for param in resnet.fc.parameters():
    param.requires_grad = True

# Training

criterion = nn.BCEWithLogitsLoss()

# note that we only pass the parameters of the final layer to the optimizer, since those
# are the only ones we are training. This is a different approach than in part (a) where
# we passed all parameters to the optimizer.
optimizer = optim.Adam(resnet.fc.parameters(), lr=1e-3)

for epoch in range(15):
    resnet.train()
    total_loss = 0

    for xb, yb in train_loader:
        optimizer.zero_grad()
        # .squeeze(1) is needed to convert the output from shape (N, 1) to (N,) so that it
        # matches the shape of the labels (N,).
        preds = resnet(xb).squeeze(1)
        loss = criterion(preds, yb)

```

```

        loss.backward()
        optimizer.step()
        total_loss += loss.item() * len(xb)

    if (epoch + 1) % 5 == 0 or epoch == 0:
        resnet.eval()
        with torch.no_grad():
            test_preds = resnet(X_test_resized).squeeze(1)
            test_pred_labels = (test_preds > 0).float()
            acc = (test_pred_labels == test_tensor_y).float().mean().item()
        print(
            f" Epoch {epoch+1:3d} | Loss: {total_loss/len(train_dataset):.4f} | Test Acc: {acc:.4f}"
        )

# Final accuracy
resnet.eval()
with torch.no_grad():
    test_preds = resnet(X_test_resized).squeeze(1)
    test_pred_labels = (test_preds > 0).float()
    acc = (test_pred_labels == test_tensor_y).float().mean().item()

print(f"\nFinal test accuracy (ResNet-18 transfer learning): {acc:.4f} ({acc*100:.1f}%)")

```

A more efficient approach is to pre-compute the features for all the training and test examples using the frozen convolutional layers, and then train a simple linear classifier on top of these features. This way, we only need to do one forward pass through the convolutional layers to get the features, and then we can train the linear classifier much more quickly.

```

"""
Part (b) optimized: Transfer learning with pre-trained ResNet-18, but with the ResNet
feature extraction done once up front instead of on every forward pass.

Since we freeze all layers except the final fc layer, the output of the feature
extractor (everything before the fc layer) is the same every epoch. We can compute
those features once and then train a simple linear classifier on top -- this is much
faster.
"""

```

```

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
import torchvision.models as models
import torchvision.transforms as transforms
import urllib.request
import os

# Download and load data

DATA_URL = "https://f000.backblazeb2.com/file/dsc-data/menu-match.npz"
DATA_PATH = "menu-match.npz"

if not os.path.exists(DATA_PATH):
    print("Downloading dataset...")
    urllib.request.urlretrieve(DATA_URL, DATA_PATH)

data = np.load(DATA_PATH)
X_train_raw = data["X_train"] # (480, 48, 48, 3)

```

```

X_test_raw = data["X_test"]
y_train_raw = data["y_train"] # strings: 'a', 'i', 's'
y_test_raw = data["y_test"]

# Convert to binary labels: 1 if Asian ('a'), 0 otherwise
y_train = (y_train_raw == "a").astype(np.float32)
y_test = (y_test_raw == "a").astype(np.float32)

# Normalize pixel values to [0, 1]
X_train = X_train_raw.astype(np.float32) / 255.0
X_test = X_test_raw.astype(np.float32) / 255.0

# Convert from (N, H, W, C) to (N, C, H, W) for PyTorch
X_train = np.transpose(X_train, (0, 3, 1, 2))
X_test = np.transpose(X_test, (0, 3, 1, 2))

print(f"Training samples: {len(y_train)}, Test samples: {len(y_test)}")
print(
    f"Asian (train): {y_train.sum():.0f}, Not Asian (train): {(1 - y_train).sum():.0f}"
)

# Prepare data for ResNet

# We will use ResNet-18 as our pre-trained model.

# ResNet was trained on ImageNet, which uses 224x224 images normalized with specific
# per-channel means and standard deviations. This means that ResNet-18 expects its
# input images to be 224x224 and normalized in the same way, so we will
# resize our 48x48 images to 224x224 and apply the same normalization.

train_tensor_x = torch.tensor(X_train)
train_tensor_y = torch.tensor(y_train)
test_tensor_x = torch.tensor(X_test)
test_tensor_y = torch.tensor(y_test)

# these are the mean and std values used for normalizing ImageNet images.
imagenet_mean = torch.tensor([0.485, 0.456, 0.406]).view(1, 3, 1, 1)
imagenet_std = torch.tensor([0.229, 0.224, 0.225]).view(1, 3, 1, 1)

resize = transforms.Resize((224, 224), antialias=True)

X_train_resized = resize(train_tensor_x)
X_test_resized = resize(test_tensor_x)

# standardize using ImageNet mean and std
X_train_resized = (X_train_resized - imagenet_mean) / imagenet_std
X_test_resized = (X_test_resized - imagenet_mean) / imagenet_std

# Precompute features

# Load a pre-trained ResNet-18 and remove the final fc layer. We'll use everything
# before it as a fixed feature extractor.
resnet = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
resnet.eval()

# Replace the fc layer with an identity so that the forward pass returns the
# 512-dimensional feature vector instead of ImageNet class logits.
resnet.fc = nn.Identity()

```



```

# Run all images through the frozen feature extractor once. This is the key
# optimization: since these features never change during training, we don't need
# to recompute them every epoch.
print("\nPrecomputing ResNet features...")
with torch.no_grad():
    train_features = resnet(X_train_resized) # (480, 512)
    test_features = resnet(X_test_resized)

print(f"Feature shape: {train_features.shape}")

# Train a linear classifier on the precomputed features

train_dataset = TensorDataset(train_features, train_tensor_y)
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)

# This is now just a single linear layer -- the same as what resnet.fc was in the other
# solution to part b, but training is now much faster because previously we had to
# evaluate the full network (including all of the convolutional layers) on every forward
# pass, whereas now we only have to evaluate the linear layer.
classifier = nn.Linear(512, 1)
criterion = nn.BCEWithLogitsLoss()
optimizer = optim.Adam(classifier.parameters(), lr=1e-3)

for epoch in range(50):
    classifier.train()
    total_loss = 0

    for xb, yb in train_loader:
        optimizer.zero_grad()
        preds = classifier(xb).squeeze(1)
        loss = criterion(preds, yb)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * len(xb)

    if (epoch + 1) % 5 == 0 or epoch == 0:
        classifier.eval()
        with torch.no_grad():
            test_preds = classifier(test_features).squeeze(1)
            test_pred_labels = (test_preds > 0).float()
            acc = (test_pred_labels == test_tensor_y).float().mean().item()
        print(
            f" Epoch {epoch+1:3d} | Loss: {total_loss/len(train_dataset):.4f} | Test Acc: {acc:.4f}"
        )

# Final accuracy
classifier.eval()
with torch.no_grad():
    test_preds = classifier(test_features).squeeze(1)
    test_pred_labels = (test_preds > 0).float()
    acc = (test_pred_labels == test_tensor_y).float().mean().item()

print(f"\nFinal test accuracy (ResNet-18 transfer learning): {acc:.4f} ({acc*100:.1f}%)")

# Save the classifier and test predictions for use in part (c)
torch.save(
    {
        "classifier": classifier.state_dict(),
        "test_features": test_features,
    }

```

```

        "test_pred_labels": test_pred_labels,
    },
    "part_b_results.pt",
)
print("Saved results to part_b_results.pt")

```

With this we are able to get the accuracy to around 91%.

- c) Perform a qualitative error analysis of your best model by looking at the test examples that it got wrong. Do you notice any patterns in the errors? For example, are there certain types of images that the model struggles with? Are there any common features among the misclassified images?

Solution:

```

"""
Part (c): Error analysis of the ResNet-18 transfer learning model.

This script loads the saved predictions from part_b_opt.py (part_b_results.pt) and
performs a qualitative error analysis by visualizing the misclassified test images.

Run part_b_opt.py first to generate the saved results.
"""

import numpy as np
import torch
import matplotlib.pyplot as plt
import urllib.request
import os

# Download and load data

DATA_URL = "https://f000.backblazeb2.com/file/dsc-data/menu-match.npz"
DATA_PATH = "menu-match.npz"

if not os.path.exists(DATA_PATH):
    print("Downloading dataset...")
    urllib.request.urlretrieve(DATA_URL, DATA_PATH)

data = np.load(DATA_PATH)
X_test_raw = data["X_test"]
y_test_raw = data["y_test"] # strings: 'a', 'i', 's'

# Convert to binary labels: 1 if Asian ('a'), 0 otherwise
y_test = (y_test_raw == "a").astype(np.float32)
test_tensor_y = torch.tensor(y_test)

# Load saved predictions from part (b)

results = torch.load("part_b_results.pt", weights_only=True)
pred_labels = results["test_pred_labels"]

# Error analysis

print("=== Part (c): Error analysis ===")

acc = (pred_labels == test_tensor_y).float().mean().item()
print(f"Test accuracy: {acc:.4f} ({acc*100:.1f}%)")

```

```

errors = (pred_labels != test_tensor_y).numpy().astype(bool)
error_indices = np.where(errors)[0]
print(f"Misclassified {len(error_indices)} out of {len(y_test)} test images.")

# Show misclassified images
if len(error_indices) > 0:
    n_show = min(len(error_indices), 12)
    ncols = min(n_show, 6)
    nrows = (n_show + ncols - 1) // ncols
    fig, axes = plt.subplots(nrows, ncols, figsize=(15, 5 * nrows / 2))
    axes = np.array(axes).flatten()
    for i, idx in enumerate(error_indices[:n_show]):
        axes[i].imshow(X_test_raw[idx])
        true = "Asian" if y_test[idx] == 1 else "Not Asian"
        predicted = "Asian" if pred_labels[idx] == 1 else "Not Asian"
        axes[i].set_title(f"True: {true}\nPred: {predicted}", fontsize=8)
        axes[i].axis("off")
    for i in range(n_show, len(axes)):
        axes[i].axis("off")
    plt.suptitle("Misclassified Test Images")
    plt.tight_layout()
    plt.savefig("error_analysis.png", dpi=150)
    print("Error analysis plot saved to error_analysis.png")

# Print breakdown of errors
false_pos = ((pred_labels == 1) & (test_tensor_y == 0)).sum().item()
false_neg = ((pred_labels == 0) & (test_tensor_y == 1)).sum().item()
print(f"False positives (predicted Asian, actually not): {false_pos}")
print(f"False negatives (predicted not Asian, actually Asian): {false_neg}")

# Show which restaurant categories were misclassified
print("\nTrue labels of misclassified images:")
for label, name in [("a", "Asian"), ("i", "Italian"), ("s", "Soup")]:
    count = (y_test_raw[errors] == label).sum()
    print(f" {name}: {count}")

```

The model misclassified 14 out of 166 test images. Here are the misclassified images:



We can see that there are many images in the data set that look quite similar but which have opposing labels. Compare, for example, the two images in the first column. One of them is labeled “Asian” and the other “Not Asian”, but in fact they appear almost identical. It seems that the main difference between them is what is on the plate. Nevertheless, the model is making what might be considered “forgivable” errors: the images it gets wrong are often quite ambiguous and could be labeled either way.